Soroush Gholizadeh Jouryabi

12454591

Brief Report — LP Praktikum Informatik 1: EEG Noise Removal Project

1. Project Context

For the LP Praktikum Informatik 1 task, the goal was to work on **EEG/BCI signal processing**, especially:

```
1. Read about EEG signal processing, EEG, and BCIs.
2. Review literature about EEG noise and artifact removal.
3. Understand which types of EEG noise/artifacts exist.
4. Investigate existing artifact-removal pipelines.
5. Implement a simple noise-removal pipeline in Python.
6. Prepare material for the final presentation.
```

So far, we completed the **literature overview part** and started the **implementation part** by creating both a Python script and a Jupyter Notebook pipeline.

---

2. Literature Overview

We reviewed four main papers. Each paper contributed a different perspective to the task.

2.1 Paper 1 — Zhang et al. 2021

EEGdenoiseNet: a benchmark dataset for deep learning solutions of EEG denoising

This paper was mainly useful for understanding **EEG denoising with deep learning** and the need for benchmark datasets.

The paper focuses mainly on two artifact types:

```
EOG – ocular artifacts
EMG – myogenic/muscle artifacts
```

EOG artifacts come from eye activity, such as eye blinks and eye movements. These artifacts often appear strongly in frontal EEG channels.

EMG artifacts come from muscle activity, such as facial, jaw, scalp, or neck muscle movements. These artifacts often have a wide frequency spectrum and can overlap with neural EEG activity.

The main contribution of this paper is **EEGdenoiseNet**, a benchmark dataset containing:

```
4514 clean EEG segments
3400 ocular artifact segments
5598 muscular artifact segments
```

The dataset allows researchers to create noisy EEG signals by mixing clean EEG with artifact segments. This is useful because, in real EEG, the true clean signal is usually unknown. With EEGdenoiseNet, denoising models can be evaluated against known ground truth.

For our project, this paper helped justify:

```
- why EEG denoising is important
- why EOG and EMG are common artifact types
- why before/after evaluation is necessary
- why PSD and frequency-domain comparisons are useful
- why deep learning is promising but requires data
```

In the slides, this paper was used to introduce **benchmark-based EEG denoising** and the idea of comparing noisy EEG, cleaned EEG, and clean reference EEG.

2.2 Paper 2 — Kumaravel et al. 2022

NEAR: An artifact removal pipeline for human newborn EEG data

This paper was useful because it describes a complete artifact-removal **pipeline**, not just one isolated method.

The paper focuses on **newborn EEG**, which is difficult because recordings are short and noisy. Newborns move unpredictably, so the EEG often contains non-stereotyped artifacts.

Important artifact sources discussed in this paper include:

```
head movement
arm movement
frowning
sucking
unstable electrode contact
bad channels
flat channels
```

A key concept from this paper is:

```
non-stereotyped artifact
```

This means the artifact does not always repeat with the same shape, timing, or spatial distribution. This is important because ICA-based methods often work better for repeated, stereotyped artifacts like eye blinks, but may struggle with unpredictable movement artifacts.

The paper proposes **NEAR**, which stands for:

```
Newborn EEG Artifact Removal
```

The NEAR pipeline includes:

```
1. Import raw EEG
2. Filtering
3. Data segmentation
4. Bad channel detection
5. Artifact removal using ASR
6. Bad channel interpolation
7. Re-referencing
8. Reporting and validation
```

Two important methods from this paper are:

```
LOF – Local Outlier Factor
ASR – Artifact Subspace Reconstruction
```

LOF is used to detect bad channels by finding channels that behave like outliers compared with other channels.

ASR is used to detect and remove or correct transient high-amplitude artifacts.

For our project, this paper helped justify the idea that EEG cleaning should be treated as a **multi-step pipeline**. In the slides, this paper was used to explain pipeline design, bad-channel handling, and ASR as an advanced method.

2.3 Paper 3 — Arpaia et al. 2025

A Systematic Review of Techniques for Artifact Detection and Artifact Category Identification in EEG from Wearable Devices

This paper was used as the broad overview paper. It is a systematic review about artifact detection in **wearable EEG**.

Wearable EEG is especially challenging because of:

```
dry or semi-wet electrodes
low channel count
reduced scalp coverage
subject mobility
uncontrolled environments
lower signal quality
```

The paper provides a broad taxonomy of artifact types:

```
ocular artifacts
muscular artifacts
movement artifacts
instrumental artifacts
cardiac artifacts
electromagnetic interference
```

Examples include:

```
eye blinks
eye movements
jaw/facial muscle activity
head/body movement
cable movement
electrode displacement
bad impedance
power-line noise
ECG contamination
```

A very important distinction from this paper is:

```
artifact detection vs artifact category identification
```

Artifact detection asks:

```
Is this EEG segment clean or contaminated?
```

Artifact category identification asks:

```
What kind of artifact is present?
Eye blink? Muscle? Movement? Electrode problem?
```

This matters because different artifact types require different removal methods.

The paper also reviews many method families:

```
filtering
ICA / BSS
wavelet methods
EMD-based methods
ASR-based methods
machine learning
deep learning
```

For our project, this paper helped build the general background slide about **types of EEG artifacts** and **common removal methods**. It also supported our conclusion that no single method removes all artifacts.

2.4 Paper 4 — Zangeneh Soroush et al. 2022

EEG artifact removal using sub-space decomposition, nonlinear dynamics, stationary wavelet transform and machine learning algorithms

This paper describes a more advanced hybrid artifact-removal method.

The method combines:

```
SOBI source separation
nonlinear feature extraction
machine learning classification
SWT wavelet denoising
EEG reconstruction
```

The artifact types considered include:

```
ECG
EMG
EOG
eye blink
white noise
```

The method first uses **SOBI**, meaning:

```
Second-Order Blind Identification
```

SOBI is a blind source separation method. It decomposes multi-channel EEG into source components.

Then, nonlinear features are extracted from each component using:

```
phase space
angle space
Poincaré-plane intersections
```

These features are used by machine learning classifiers such as:

```
MLP
KNN
Naïve Bayes
SVM
ensemble classifiers
```

The classifiers decide whether a component is neural or artifactual.

Instead of simply deleting artifact components, the paper uses:

```
SWT – Stationary Wavelet Transform
```

SWT is used to denoise artifact components while trying to preserve neural information that may have leaked into those components.

This paper was useful because it showed that:

```
- BSS alone is not enough
- artifact components must be identified carefully
- deleting full components may remove neural signal
- wavelet denoising can preserve more information
- evaluation should include time-domain and frequency-domain
measures
```

In the slides, this paper was used as the advanced technical method example.

---

3. Overall Literature Conclusion

From the four papers, the main conclusion is:

```
EEG artifact removal is not one universal algorithm.
It is a pipeline problem.
```

Different artifacts require different methods.

For example:

```
slow drift → high-pass filtering
high-frequency noise → low-pass filtering
50 Hz power-line noise → notch filtering
eye blinks → ICA/BSS or EOG-based correction
movement artifacts → ASR or segment rejection
bad channels → detection and interpolation
complex artifacts → ML, wavelets, or deep learning
```

This motivated our implementation of a simple, explainable Python/MNE pipeline.

---

4. Implementation Overview

We created two implementation versions:

```
1. eeg_noise_removal_pipeline.py
2. eeg_noise_removal_pipeline_notebook.ipynb
```

The Jupyter Notebook version was created because it is easier to run step by step, inspect outputs, and use figures for the final presentation.

The implementation is a **simple baseline pipeline** inspired by the literature, not a full reproduction of NEAR, EEGdenoiseNet, or SOBI-SWT.

5. Code Structure and Explanation

5.1 Configuration Section

The configuration section defines the input EEG file and preprocessing parameters.

Example:

```
INPUT_FILE = Path("data/S001R01.edf")
OUTPUT_DIR = Path("results_notebook")

L_FREQ = 1.0
H_FREQ = 40.0
NOTCH_FREQ = 50.0

RUN_ICA = False
SAVE_CLEANED = True
```

This means:

```
Input file: data/S001R01.edf
Output folder: results_notebook
Band-pass filter: 1–40 Hz
Notch filter: 50 Hz
ICA: disabled for the first run
Save cleaned EEG: yes
```

5.2 Loading EEG Data

The function load_eeg() loads EEG files using MNE-Python.

It supports formats such as:

```
.edf
.bdf
.set
.fif
.vhdr
```

For our current run, we used:

```
S001R01.edf
```

The notebook checks whether the file exists before loading it. This caused a path error at first, but it was fixed by changing:

```
INPUT_FILE = Path("data/your_eeg_file.edf")
```

to:

```
INPUT_FILE = Path("data/S001R01.edf")
```

5.3 Metadata Report

After loading the file, the code saves basic metadata using:

```
save_basic_report(raw, OUTPUT_DIR)
```

This creates:

```
metadata_report.json
```

The metadata includes:

```
number of channels
channel names
channel types
sampling frequency
recording duration
reference information
```

This is useful because we can describe the EEG recording in the final report or presentation.

5.4 Time-Domain Plot

The function:

```
plot_raw_segment()
```

plots a short segment of EEG in the **time domain**.

The time domain means:

```
x-axis = time
y-axis = voltage/amplitude
```

This helps visually inspect:

```
large eye-blink-like waves
sudden jumps
movement artifacts
flat channels
very noisy channels
```

The raw output file is:

```
01_raw_time_domain.png
```

After filtering, the cleaned output is:

```
03_cleaned_time_domain.png
```

5.5 PSD / Frequency-Domain Plot

The function:

```
plot_psd()
```

plots the **Power Spectral Density**, or PSD.

PSD shows how much signal power exists at each frequency.

The frequency domain means:

```
x-axis = frequency in Hz
y-axis = signal power
```

This is useful for detecting:

```
50 Hz power-line noise
high-frequency muscle noise
slow low-frequency drift
changes in EEG frequency bands
```

The raw PSD output is:

```
02_raw_psd.png
```

The cleaned PSD output is:

```
04_cleaned_psd.png
```

5.6 Band-Pass Filtering

The function:

```
apply_basic_filters()
```

applies the main filtering pipeline.

The band-pass filter is:

```
1–40 Hz
```

This means:

```
frequencies below 1 Hz are reduced
frequencies above 40 Hz are reduced
frequencies between 1 and 40 Hz are preserved
```

The high-pass part helps reduce:

```
slow baseline drift
very slow movement-related changes
sweat/electrode drift
```

The low-pass part helps reduce:

```
high-frequency noise
some EMG/muscle activity
unnecessary high-frequency components
```

5.7 Notch Filtering

The notch filter is applied at:

```
50 Hz
```

This targets power-line interference, which is common in Europe.

The idea is:

```
remove/reduce one specific frequency while keeping the rest of the signal mostly unchanged
```

This connects directly to the literature discussion about electromagnetic interference and power-line noise.

5.8 Optional ICA

The notebook also includes optional ICA:

```
RUN_ICA = False
```

ICA stands for:

```
Independent Component Analysis
```

ICA is a component-based method. It separates EEG into components, and some components may correspond to artifacts such as eye blinks or ECG.

For the first run, ICA was kept disabled because filtering and PSD comparison are enough for a basic implementation. ICA can be enabled later by changing:

```
RUN_ICA = True
```

The code can then try to detect EOG/ECG-related components if suitable channels exist.

5.9 Raw vs Cleaned PSD Comparison

The most important output for the implementation is:

```
05_psd_raw_vs_cleaned.png
```

This plot compares the raw EEG PSD and cleaned EEG PSD.

It helps answer:

```
Did filtering reduce unwanted frequency components?
Was high-frequency noise reduced?
Was 50 Hz noise reduced?
Was the EEG frequency content changed?
```

This figure is especially useful for the final presentation because it directly shows the effect of preprocessing.

5.10 Saving Cleaned Data and Report

The notebook saves the cleaned EEG as:

```
cleaned_eeg_raw.fif
```

It also saves a processing report:

```
pipeline_report.json
```

The report contains:

```
input file path
metadata
filter settings
notch frequency
ICA settings
output file names
```

This makes the pipeline more reproducible.

---

6. Current Repository Content

The project contains:

```
eeg_noise_removal_pipeline.py
eeg_noise_removal_pipeline_notebook.ipynb
README.md
implementation_explanation.md
data/S001R01.edf
results_notebook/
```

The GitHub repository link consisting of the regarding mentioned files is:

https://github.com/Sorooooush/LP-1